

Đại học quốc gia TP.HCM
Trường đại học công nghệ thông tin



Ngành: Khoa học máy tính

Môn học: IT005.P21.CTTN

Thực hành TCP UDP

Sinh viên:

Trần Quang Trường - MSSV: 24521901

1 UDP

1.1 Tính checksum thủ công của các gói tin UDP có Arrival Time: Jan 1, 1970 08:00:13.398602000 +08

Theo dữ liệu đầu bài, ta dễ dàng biết được gói tin cần tính checksum là gói tin số 256.

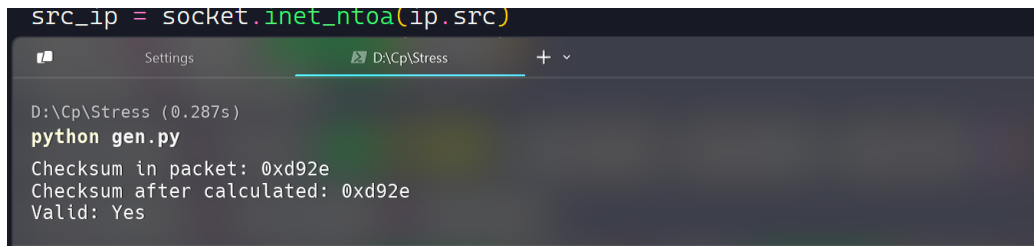
Ta có thể tính checksum thông qua đoạn code python dưới đây:

```
1 import socket, struct, dpkt, array
2
3 def checksum(pkt: bytes, udp=False) -> int:
4     if len(pkt) % 2 == 1:
5         pkt += b'\x00'
6         s = sum(array.array("H", pkt))
7         s = (s >> 16) + (s & 0xffff)
8         s += s >> 16
9         s = ~s & 0xffff
10        if struct.pack("H", 1) == b"\x01\x00":
11            s = (((s >> 8) & 0xff) | (s << 8)) & 0xffff
12        if udp and s == 0x0000:
13            return 0xffff
14        return s
15
16 with open("udp.pcap", "rb") as f:
17     pcap = dpkt.pcap.Reader(f)
18     packets = list(pcap)
19
20 ts, data = packets[255]
21 eth = dpkt.ethernet.Ethernet(data)
22 ip = eth.data
23 udp = ip.data
24
25 src_ip = socket.inet_ntoa(ip.src)
26 dst_ip = socket.inet_ntoa(ip.dst)
27 udp_data = udp.data
28
29 udp_header = struct.pack('!HHHH', udp.sport, udp.dport, udp.
30     ulen, 0)
31
32 udp_segment = udp_header + udp_data
33
34 pseudo_header = socket.inet_aton(src_ip) + socket.inet_aton(
35     dst_ip) + struct.pack('!BBH', 0, ip.p, udp.ulen)
```

```

35 checksum_value = checksum(pseudo_header + udp_segment, udp=
    True)
36
37 print("Checksum in packet:", f"0x{udp.sum:04x}")
38 print("Checksum after calculated:", f"0x{checksum_value:04x}"
    )
39 print("Valid:", "Yes" if (udp.sum == checksum_value) else "No"
    ")

```



```

src_ip = socket.inet_ntoa(ip.src)

D:\Cp\Stress (0.287s)
python gen.py
Checksum in packet: 0xd92e
Checksum after calculated: 0xd92e
Valid: Yes

```

Figure 1: Kết quả sau khi chạy chương trình

Kết luận, checksum của gói tin cần tìm bằng 0xd92e.

1.2 Có 2 người bạn truyền gói tin cho nhau thông qua udp, biết là mỗi gói tin udp truyền đi 1 kí tự, trong lúc đó có ai đó gửi thêm những gói tin không liên quan và bị đánh dấu bởi security flags, hãy cố gắng lọc các gói tin đó ra và tìm thấy nội dung được truyền đi.

Dễ thấy, có một số packet có `flag = 0x0` là có gói tin liên quan nên ta sẽ lọc chúng bằng đoạn code sau đây:

```

1 import dpkt
2 from scapy.all import rdpcap, IP, UDP
3
4 with open("udp.pcap", "rb") as f:
5     pcap = dpkt.pcap.Reader(f)
6     packets = list(pcap)
7
8 packets = rdpcap("udp.pcap")
9

```

```

10 filter_files = []
11
12 for pkt in packets:
13     if pkt.haslayer(IP) and pkt.haslayer(UDP):
14         ip_layer = pkt[IP]
15         udp_layer = pkt[UDP]
16
17         if ip_layer.flags == 0:
18             payload = bytes(udp_layer.payload).decode(errors=
19                 "ignore").strip()
20
21             if payload:
22                 filter_files.append(payload)
23
24 hidden_message = ''.join(filter_files)
25 print(f'Hidden message is {hidden_message}')

```

```

D:\Cp\Stress (1.033s)
python gen.py
Hidden message is 3V1L_817_3xf1l7R4710N_4_7H3_W1N_51D43c8000034d0c

D:\Cp\Stress  Pair Ctrl I  Dispatch Beta Ctrl Shift I
python test.py

```

Figure 2: Kết quả sau khi chạy chương trình

Vậy thông điệp ẩn ở đây là "3V1L_817_3xf1l7R4710N_4_7H3_W1N_51D43c8000034d0c". Bằng cách thay đổi một số kí tự số thành chữ có nét tương tự thể suy đoán được nội dung của thông điệp là "EVIL BIT EXFILTRATION FOR THE WIN SIDE".

2 TCP

2.1 TCP sender

2.1.1 Sự kiện: Nhận dữ liệu từ tầng ứng dụng

Xét các gói sau:

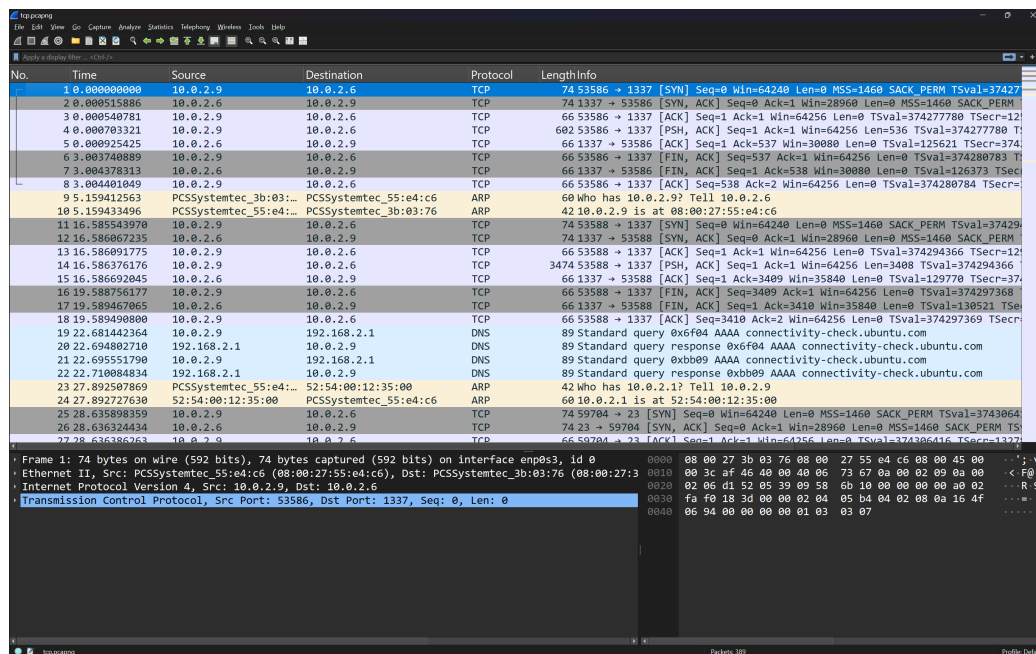


Figure 3: File tcp.pcapng

- Gói 3: Tạo segment với seq = 1, ACK = 1. Bắt đầu tính thời gian tại đây.
- Gói 4: Gửi lại gói 3 (do chưa nhận được ACK hoặc timeout).

Tính RTT (Round Trip Time):

- Gói 3 gửi lúc: 0.000540781s
- Gói 7 xác nhận rõ ràng (ACK = 538) lúc: 3.004378313s
- $RTT = 3.004378313 - 0.000540781 \text{ s}$

Phiên TCP tiếp theo:

- Gói 13: seq = 1 (kết nối mới)
- Gói 17: ACK = 3410, thời gian = 19.589467065s
- Gói 13 gửi lúc: 16.586091775s
- $RTT = 19.589467065 - 16.586091775 \text{ s}$

2.1.2 Sự kiện: timeout

- $\text{Timeout} = 16.586091775 - 0.000540781 = 16.585550994\text{s}$

2.1.3 Sự kiện: nhận được ACK

TCP Receiver: ACK generation [RFC 5681]

2.1.4 Nhận được segment đúng với STT đang chờ. Tất cả segment trước đó đã được ACK.

- Gói 3 (Seq = 1): gửi dữ liệu
- Gói 7 (ACK = 538): xác nhận toàn bộ dữ liệu, cho thấy đã nhận đầy đủ

2.1.5 Nhận được segment đúng với STT đang chờ, một segment chưa được ACK.

Không có.

2.1.6 Nhận được segment không đúng thứ tự (STT cao hơn).

Không có.

2.1.7 Nhận được segment trong khoảng bị trống (giữa STT đang chờ và STT nhận được trước đó).

Không có.